EE5203 L11 Lab Guide

The disassembly view — your C, as the machine sees it - Basri Erdogan

Mission: Run your own L10 project one instruction at a time: the load-compute-store of your ADC math, the branch of your night-light `if`, the `bl` into a call—every step predicted first, every prediction checked in the Registers view.

Prepare before class

- ☐ Re-read the theory slides up to "BL and the way home."
- ☐ Bring your working L10 project. No new hardware—today's instrument is the debugger.
- ☐ Know the answer to: after `movs r3, #5` then `lsls r3, r3, #2`, what is `r3`?
- ☐ Know which register carries a function's first argument—and its result.

Learning objectives

By checkoff time, you can:

1. drive the Disassembly view and single-instruction stepping;
2. map a C statement to its `-O0` instructions, load-compute-store;
3. predict `r0`–`r3`, `sp`, `lr`, `pc`, and the flags before each step;
4. explain what `-O2` did to one of your functions.

Setup

Item	Where
Project	your `L10_ADC`, debug build (`-O0` — the default Debug config)
Views	Disassembly View (right-click → Open Disassembly View), PERIPHERALS/SFR, MEMORY
Stepping	debug toolbar: **Instruction Stepping Mode** (the `i→` button)
Registers	expand *Core registers*; changed values highlight after each step

Start a debug session, let it pause at `main`, dock the Disassembly view beside the editor. With Instruction Stepping Mode on, *Step Into* moves one instruction; the highlighted line tracks `pc`.

Checkpoint A - One instruction at a time

Breakpoint on your millivolt line: `mv_pot = (raw_pot * 3300u) / 4095u;`. Step through it by instructions.

Evidence for Checkpoint A: you point at the load-compute-store shape in the listing; before each step you name the register about to change, then the Registers view confirms it; `pc` advances by 2—and by 4 across one `.w` instruction you point out.

Checkpoint B - The if and its flags

Breakpoint on a comparison of yours (night-light threshold, or the servo clamp). Step to the `cmp`.

Evidence for Checkpoint B: `xpsr` before and after the `cmp`—which flags changed; the branch that follows (`bls`/`bhi`/`bcs`…) matched to your C condition, and the *unsigned* family explained from your `uint32_t` types; both paths demonstrated live (turn the knob / cover the LDR, run to the breakpoint again).

Checkpoint C - The call, traced

Breakpoint just before `adc_read(ADC_CHANNEL_0)`. Step into it by instructions.

Evidence for Checkpoint C: the argument appears in `r0` before the `bl`; the `bl` writes `lr` and teleports `pc`; the `push {r7, lr}` drops `sp` by 8—you find the saved return address at the new `sp` in the Memory view; the result rides home in `r0`; `pop {…, pc}` lands execution back at the call site.

Then the optimizer: in a *copy* of the project (or a second build config), set *Properties → C/C++ Build → Settings → Optimization* to `-O2`, rebuild, and find the same function in the Disassembly view.

Individual extension

Pick one, predict first:

- **The one-instruction HAL:** step into `__HAL_TIM_SET_COMPARE(...)` at `-O0`—show that the "function" is a single `str`. Where did it go at `-O2`?

- **The literal pool:** find the `.word` holding a peripheral address after one of your functions, and the `ldr` with `[pc, #…]` that fetches it.
- **The vanished call:** write `uint32_t twice(uint32_t x){return 2*x;}`, call it with a constant, build at `-O2`—find what remains of it.

Acceptance checklist

☐ Instruction stepping shown; `pc`, `sp`, `lr` roles explained live.
☐ One C statement mapped instruction-by-instruction, predictions first.
☐ `cmp` + branch matched to your `if`; flag changes shown in `xpsr`.
☐ A call traced end to end: `r0` in, `lr`, `sp`, `r0` out, `pop {…, pc}`.
☐ The same function compared at `-O0` vs `-O2`, differences explained.

Individual oral check

- What is the difference between `r3` and `[r3]`?
- `add` versus `adds`—what changes, and who reads it?
- What exactly does `pop {r7, pc}` do?
- Why did your `uint32_t` comparison compile to `bls`, not `ble`?
- The program hard-faults at `pc = 0x08001a32`. What is your first move?

Submit

Submit two screenshots—Disassembly + Registers mid-step at Checkpoint A, and the `-O0` vs `-O2` listing of your compared function—plus a six-row trace table for your millivolt statement: instruction, register changed, predicted value, observed value.

If you are stuck

Walk the chain: paused in a live debug session? Then: **Disassembly view empty** — the target is running; pause first; **steps move whole C lines** — Instruction Stepping Mode is off; **registers greyed out** — same cause; **cannot find your function at `-O2`** — it was inlined or folded into a constant: look inside its caller; **the highlighted source line jumps around at `-O2`** — that is the optimizer reordering, not a debugger bug; compare against the `-O0` build.